ACADEMIC CENTRAL

PRACTICE EXAMINATION

UNIVERSITY · 3RD YEAR

THEORY OF COMPUTATION

PAPER 1 · PRACTICE A · MARKING GUIDELINE

MARKS: 100

Marking guideline for the Academic Central University · 3rd year Theory of Computation paper 1 · practice a.

Written by Academic Central for practice, following third-year Computer Science syllabuses at South African universities (theory of computation: automata, computability and complexity). It is not a university memorandum.

How to mark: each ✓ is one mark. Accept any correct automaton, grammar, expression or proof, even if it differs from the model answer (state names do not matter). Check a different answer by testing it on a few strings that should be accepted and rejected.

QUESTION 1

1.1 B ✓

1.2 A ✓

1.3 C ✓

1.4 C ✓

1.5 A ✓

1.6 B ✓

1.7 C ✓

1.8 A ✓

1.9 A ✓

1.10 A ✓

(10)

[10]

QUESTION 2

2.1.1 Q = {r0, r1, r2} and Σ = {0, 1} ✓ q_0 = r0 and F = {r0} ✓ (4)

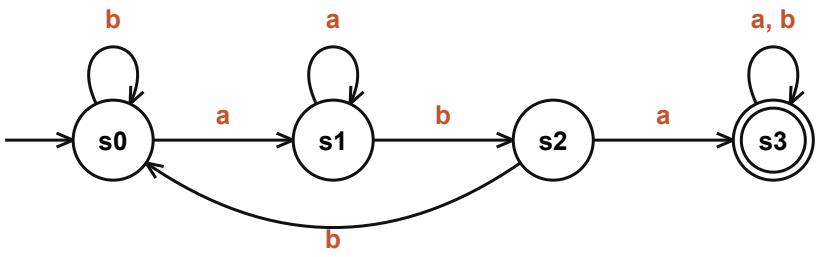
δ	0	1
r0	r0	r1
r1	r2	r0
r2	r1	r2

Table ✓✓

2.1.2 1001: r0 → r1 → r2 → r1 → r0 ✓; ends in an accept state, so it is accepted ✓ (3)
1011: r0 → r1 → r2 → r2 → r2; rejected ✓

2.1.3 Binary numbers whose value is divisible by 3 ✓ (the state is the remainder after dividing by 3), including the empty string ✓ (2)

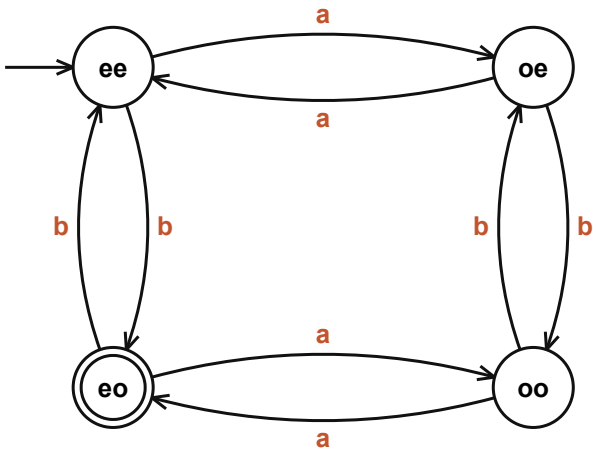
2.2.1



s0: nothing found yet; s1: seen a; s2: seen ab; s3: seen aba (stays there). Start and accept state ✓ s0, s1 and s2 transitions ✓✓ s3 loops on a and b ✓

(4)

2.2.2



The state name shows whether the numbers of a's and b's read so far are even (e) or odd (o). Four states ✓ a-transitions ✓ b-transitions ✓ start ee, accept eo ✓

(4)

2.3 Keep the same states and transitions ✓ and swap the accept and non-accept states (F becomes Q − F) ✓
So the complement of a regular language is also regular: regular languages are closed under complement ✓ (3)

[20]

QUESTION 3

3.1.1 Yes ✓: p –0→ p –1→ p –1→ q –0→ r, and r is an accept state (the second-last symbol is 1) ✓ (2)

3.1.2 (6)

DFA state	0	1
→ {p}	{p}	{p, q}
{p, q}	{p, r}	{p, q, r}
* {p, r}	{p}	{p, q}
* {p, q, r}	{p, r}	{p, q, r}

(→ start state, * accept state)
Start state {p} ✓ row {p} ✓ row {p, q} ✓ rows {p, r} and {p, q, r} ✓✓ accept states are the sets that contain r ✓
(A correct state diagram is also accepted.)

3.2.1 a(a ∪ b)*b ✓✓ (2)

3.2.2 (0 ∪ 10)*(ε ∪ 1) ✓✓ (2)
(Also accept, e.g., 0*(100*)*(ε ∪ 1) or (ε ∪ 1)(0 ∪ 01)*.)

3.3 Any TWO: an NFA is often much smaller and easier to design than a DFA ✓; it makes proofs easy (e.g. closure under union, concatenation and star) ✓; a regular expression converts directly into an NFA (3)
Price: the DFA can have up to 2^n states for an NFA with n states ✓

[15]

QUESTION 4

4.1 If A is a regular language, there is a number p (the pumping length) ✓ such that every string s in A with |s| ≥ p can be divided into three pieces, s = xyz ✓, so that: (4)
1. $xy^iz \in A$ for every i ≥ 0 ✓
2. |y| > 0 and 3. |xy| ≤ p ✓

4.2 Assume, for a contradiction, that L is regular, with pumping length p ✓ (6)
Choose s = 0^p1^p; s is in L and |s| = 2p ≥ p ✓
For any division s = xyz with |xy| ≤ p, x and y lie inside the first p symbols, so y consists of 0s only ✓: y = 0^k with k ≥ 1 because |y| > 0 ✓
Pump with i = 2: $xy^2z = 0^{p+k}1^p$ has more 0s than 1s, so it is not in L ✓
This contradicts the pumping lemma, so L is not regular ✓

4.3 Suppose EQ were regular. 0*1* is regular and regular languages are closed under intersection, so EQ ∩ 0*1* would be regular ✓ (3)
But EQ ∩ 0*1* = $\{0^n1^n \mid n \geq 0\}$ ✓, which is not regular (4.2) ✓
So EQ is not regular.

4.4 Yes ✓. A finite language $\{w_1, \ldots, w_k\}$ is described by the regular expression $w_1 \cup w_2 \cup \ldots \cup w_k$ (or a DFA can be built with a separate path for each string) ✓ (2)

[15]

QUESTION 5

5.1.1 S ⇒ SS ⇒ aSbS ✓ ⇒ aaSbbS ⇒ aabbS ✓ ⇒ aabbaSb ⇒ aabbab ✓ (3)

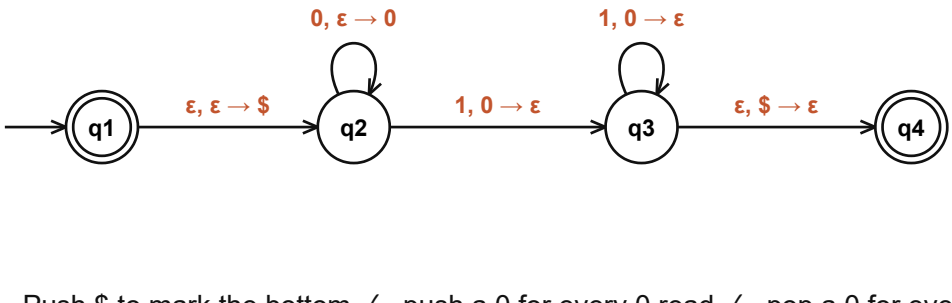
5.1.2 The string ab has two different leftmost derivations (so two different parse trees) ✓: (3)
S ⇒ aSb ⇒ ab ✓
S ⇒ SS ⇒ aSbS ⇒ abS ⇒ ab ✓
A string with two leftmost derivations means G is ambiguous. (Accept other strings, e.g. ε: S ⇒ ε and S ⇒ SS ⇒ S ⇒ ε.)

5.1.3 All strings of correctly matched (balanced) brackets, with a as "(" and b as ")" ✓ (1)

5.2.1 S → aSbb | ε ✓✓ (2)

5.2.2 S → 0S0 | 1S1 | 0 | 1 | ε ✓✓ (2)

5.3



Push $ to mark the bottom ✓ push a 0 for every 0 read ✓ pop a 0 for every 1 read ✓✓ accept when $ is on top, and accept ε at the start ✓ (5)

5.4 The syntax of a programming language has nested structures (brackets, blocks, if–else, expressions) that a CFG describes exactly ✓, and efficient parsers (e.g. LL and LR) can be built straight from the grammar ✓ (3)
A CFG cannot check, e.g., that every variable is declared before it is used, or that types match ✓

5.5 $\{a^n b^n c^n \mid n \geq 0\}$ ✓ (or $\{ww \mid w \in \{0, 1\}^*\}$) (1)

[20]

QUESTION 6

6.1.1 q0 011 ⊢ 1 q0 11 ⊢ 10 q0 1 ⊢ 100 q0 ⊔ ✓ (3)
⊢ 10 q1 0 ⊢ 1 q1 00 ⊢ q1 100 ✓
⊢ q1 ⊔100 ⊢ ⊔ q_{acc} 100 ✓

6.1.2 It flips every bit (0 becomes 1 and 1 becomes 0), moves back to the start of the string and accepts ✓ (1)

6.2 A language is Turing-recognisable if some TM accepts every string in it; on strings not in it the TM may reject or loop forever ✓ (3)
A language is decidable if some TM accepts every string in it and rejects every string not in it, so it always halts ✓
Every decidable language is recognisable, but not the other way around (e.g. A_{TM}) ✓

6.3 Assume a TM H decides A_{TM}: H accepts ⟨M, w⟩ if M accepts w and rejects otherwise ✓ (6)
Build a TM D that, on input ⟨M⟩, runs H on ⟨M, ⟨M⟩⟩ ✓ and does the opposite: D accepts if H rejects, and rejects if H accepts ✓
Now run D on its own description ⟨D⟩ ✓
D accepts ⟨D⟩ exactly when H says D does not accept ⟨D⟩, i.e. exactly when D does not accept ⟨D⟩ ✓
This is a contradiction, so H cannot exist and A_{TM} is undecidable ✓

6.4 P: problems that a deterministic TM decides in polynomial time ✓ (5)
NP: problems whose solutions (certificates) can be checked in polynomial time, or that a nondeterministic TM decides in polynomial time ✓
NP-complete: the problem is in NP ✓ and every problem in NP reduces to it in polynomial time ✓
Example: SAT (or 3-SAT, CLIQUE, HAMPATH, VERTEX-COVER, SUBSET-SUM) ✓

6.5 Every NP problem reduces in polynomial time to the NP-complete problem ✓; doing the reduction and then running the polynomial algorithm solves every NP problem in polynomial time, so NP ⊆ P, and since P ⊆ NP, P = NP ✓ (2)

[20]

TOTAL: 100

END OF MARKING GUIDELINE